DHT 노드 및 라우팅 테이블 분석

개요

rqbit의 DHT 구현은 Kademlia 알고리즘을 기반으로 하며, 분산 해시 테이블을 통해 피어 검색과 토렌트 정보 공유를 수행합니다. 이 문서는 DHT 노드 구조, 라우팅 테이블의 k-bucket 구현, 그리고 노드 상태 관리 메커니즘을 상세히 분석합니다.

DHT 노드 구조 분석

DhtState 구조체

DHT의 핵심은 `DhtState` 구조체로, 전체 DHT 노드의 상태를 관리합니다:

```
pub struct DhtState {
id: Id20, // 노드의 고유 식별자 (160비트)
next_transaction_id: AtomicU16, // 트랜잭션 ID 생성기
// 진행 중인 요청 추적
inflight_by_transaction_id: DashMap<(u16, SocketAddr), OutstandingRequest>,
// IPv4/IPv6 라우팅 테이블
routing_table_v4: RwLock<RoutingTable>,
routing_table_v6: RwLock<RoutingTable>,
listen_addr: SocketAddr, // 리스닝 주소
rate_limiter: RateLimiter, // 요청 속도 제한
worker_sender: UnboundedSender<WorkerSendRequest>, // 워커 스레드 통신
```

```
cancellation_token: CancellationToken, // 종료 신호
peer_store: PeerStore, // 피어 정보 저장소
}
```

노드 식별자와 거리 계산

DHT에서 노드 간의 거리는 XOR 메트릭을 사용하여 계산됩니다:

```
// Id20 구조체에서 거리 계산
impl Id20 {
pub fn distance(&self, other: &Id20) -> Id20 {
let mut result = [0u8; 20];
for i in 0..20 {
result[i] = self.0[i] ^ other.0[i];
}
Id20::new(result)
}
}
```

이 거리 메트릭은 Kademlia 알고리즘의 핵심으로, 노드들을 트리 구조로 조직화하는 데 사용됩니다.

라우팅 테이블 구현 분석

RoutingTable 구조체

라우팅 테이블은 알려진 노드들을 효율적으로 관리하기 위한 자료구조입니다:

```
pub struct RoutingTable {
id: Id20, // 자신의 노드 ID
size: usize, // 저장된 노드 수
buckets: BucketTree, // k-bucket 트리 구조
}
```

k-bucket 트리 구현

rqbit은 전통적인 고정 크기 버킷 배열 대신 동적 트리 구조를 사용합니다:

```
pub struct BucketTree {
data: Vec<BucketTreeNode>, // 트리 노드들
size: usize, // 총 노드 수
max_size: usize, // 최대 노드 수 (기본값: 512)
}

struct BucketTreeNode {
bits: u8, // 이 노드가 담당하는 비트 수
start: Id20, // 범위 시작
end_inclusive: Id20, // 범위 끝 (포함)
data: BucketTreeNodeData, // 리프 또는 내부 노드 데이터
}

enum BucketTreeNodeData {
```

```
Leaf(LeafBucket), // 리프: 실제 노드들 저장
LeftRight(usize, usize), // 내부 노드: 자식 인덱스
}
```

버킷 분할 알고리즘

버킷이 가득 찰 때 (8개 노드), 다음 조건에서 분할이 발생합니다:

1. 자신의 ID가 해당 버킷 범위 내에 있어야 함
2. 최대 노드 수에 도달하지 않았어야 함
3. 나쁜 노드로 교체할 수 없는 경우

```
fn compute_split_start_end(
start: Id20,
end_inclusive: Id20,
bits: u8,
) -> ((Id20, Id20), (Id20, Id20)) {
let changing_bit = 160 - bits;
let new_left_end = {
let mut c = end_inclusive;
c.set_bit(changing_bit, false);
c
};
let new_right_start = {
let mut c = start;
```

```
c.set_bit(changing_bit, true);

c

};

((start, new_left_end), (new_right_start, end_inclusive))

}
```

노드 상태 관리

RoutingTableNode 구조체

각 노드는 다음 정보를 유지합니다:

```
pub struct RoutingTableNode {

id: Id20, // 노드 ID

addr: SocketAddr, // 네트워크 주소

last_request: Option<Instant>, // 마지막 요청 시간

last_response: Option<Instant>, // 마지막 응답 시간

last_query: Option<Instant>, // 마지막 쿼리 수신 시간

errors_in_a_row: usize, // 연속 오류 횟수

}
```

노드 상태 분류

노드는 다음 4가지 상태로 분류됩니다:

```rust
pub enum NodeStatus {
Good, // 양호: 15분 내 응답 또는 쿼리
Questionable, // 의심스러운: 15분 이상 비활성
Bad, // 나쁨: 2회 이상 연속 오류
Unknown, // 알 수 없음: 초기 상태
}
```

상태 결정 로직:

```rust
impl RoutingTableNode {
pub fn status(&self, now: Instant) -> NodeStatus {
match (self.last_request, self.last_response, self.last_query) {
// 2회 이상 연속 오류 시 나쁨
(Some(_), _, _) if self.errors_in_a_row >= 2 => NodeStatus::Bad,
// 15분 내 응답이 있으면 양호
(Some(_), Some(last_incoming), _) | (Some(_), Some(_), Some(last_incoming))
if now - last_incoming < INACTIVITY_TIMEOUT => NodeStatus::Good,
// 15분 이상 비활성이면 의심스러운
(last_outgoing, _, Some(last_incoming)) | (last_outgoing, Some(last_incoming), _
if now - last_incoming > INACTIVITY_TIMEOUT
&& last_outgoing.map(|e| now - e > INACTIVITY_TIMEOUT).unwrap_or(true) =>
NodeStatus::Questionable,
_ => NodeStatus::Unknown,
```

```
}
}
}
```

노드 추가 및 관리

노드 추가 과정

새 노드를 라우팅 테이블에 추가할 때의 과정:

1. **기존 노드 확인**: 이미 존재하는 노드인지 확인
2. **나쁜 노드 교체**: 나쁜 상태의 노드가 있으면 교체
3. **공간 확인**: 버킷에 여유 공간이 있으면 추가
4. **분할 시도**: 자신의 ID 범위 내이고 최대 크기 미달이면 버킷 분할
5. **무시**: 조건에 맞지 않으면 무시

```
pub enum InsertResult {
WasExisting, // 이미 존재함
ReplacedBad(RoutingTableNode), // 나쁜 노드 교체
Added, // 새로 추가됨
Ignored, // 무시됨
}
```

거리 기반 정렬

특정 ID로부터의 거리에 따라 노드들을 정렬:

```
pub fn sorted_by_distance_from(&self, id: Id20, now: Instant) -> Vec<&RoutingTab
let mut result = Vec::with_capacity(self.size);
for node in self.buckets.iter() {
result.push(node);
}
result.sort_by_key(|n| {
let status = match n.status(now) {
NodeStatus::Good => 0,
NodeStatus::Questionable => 1,
NodeStatus::Unknown => 2,
NodeStatus::Bad => 3,
};
(status, id.distance(&n.id))
});
result
}
```

이 정렬은 상태가 좋은 노드를 우선하고, 같은 상태 내에서는 거리가 가까운 노드를 우선합니다.

생존성 확인 과정

주기적 상태 확인

DHT는 다음과 같은 방식으로 노드의 생존성을 확인합니다:

1. **비활성 타임아웃**: 15분 이상 비활성 노드를 의심스러운 상태로 변경
2. **오류 추적**: 연속 2회 이상 오류 시 나쁜 상태로 변경
3. **주기적 쿼리**: 의심스러운 노드에 대해 ping 쿼리 전송

노드 상태 업데이트

노드와의 상호작용에 따른 상태 업데이트:

```
impl RoutingTableNode {
pub fn mark_outgoing_request(&mut self, now: Instant) {
self.last_request = Some(now);
}

pub fn mark_response(&mut self, now: Instant) {
self.last_response = Some(now);
if self.last_request.is_none() {
self.last_request = Some(now);
}
self.errors_in_a_row = 0; // 오류 카운터 리셋
}

pub fn mark_error(&mut self) {
self.errors_in_a_row += 1; // 오류 카운터 증가
}
```

```
pub fn mark_last_query(&mut self, now: Instant) {

self.last_query = Some(now);

}

}
```

성능 최적화

메모리 효율성

- **동적 트리 구조**: 필요에 따라 버킷을 분할하여 메모리 사용량 최적화
- **참조 카운팅**: Arc를 사용한 효율적인 데이터 공유
- **압축된 노드 정보**: CompactNodeInfo를 통한 네트워크 전송 최적화

동시성 처리

- **RwLock**: 읽기 작업이 많은 라우팅 테이블에 적합한 동기화
- **DashMap**: 높은 동시성을 위한 해시맵 구현
- **원자적 연산**: 트랜잭션 ID 생성 등에 AtomicU16 사용

이러한 구현을 통해 rqbit의 DHT는 효율적이고 확장 가능한 분산 해시 테이블을 제공하며, Kademlia 알고리즘의 핵심 특성을 잘 구현하고 있습니다.